

Command Design Pattern

Lecture notes



Command: motivation

- Request receivers can receive many kinds of request
 - Often requires a large selection statement to work out the appropriate response
 - The request is usually delegated to other classes
 - Knowledge of which delegate class is needed for each request is built into the receiver of the request
- It can be desirable to remove this knowledge from the request receiver
 - Simply instantiate a delegate and execute it...
 - ...without knowing either the actual request type or the delegate type

Command: in practice

- The Command design pattern...
 - Encapsulates a command (response to a request) as an object
 - Permits the issuing of commands without knowing about the request or its recipient
- For example, the methods in façade classes ...
 - Could be re-written as commands
 - The façade class simply instantiates a command (using a factory method), and executes it...
 - ... without knowing either the request type or the command type

Activity



Coding

- Review the example code and watch out for:
 - how the starting point project has a very large `Main()` and associated methods
 - how the project with patterns has cleaner code because it uses commands
- If you have any questions, ask

Projects: `LectureExample_StartingPoint`
and: `LectureExample_WithPatterns`